

# On the Impact of ChatGPT on Teaching and Studying Software Engineering

Benedikt Zönnchen\*, Veronika Thurner<sup>§</sup> and Axel Böttcher<sup>§</sup>

*\*Munich Center for Digital Sciences and AI*

*§Department of Computer Science and Mathematics*

*Hochschule München University of Applied Sciences, Munich, Germany*

zoennchen.benedikt@hm.edu, veronika.thurner@hm.edu, axel.boettcher@hm.edu

**Abstract**—AI-systems that are based on large language models, such as ChatGPT, have quickly increased their prowess over the last year, and at the same time became readily available. As of now, many disciplines gain experience in using tools such as ChatGPT in a professional setting – and software engineering is no exception. Just as with any new kind of tooling, it is to be expected that in the era of ChatGPT, some traditional skills of the discipline will become rather obsolete, while at the same time new skill sets emerge that will be required from future professionals. Therefore, as educators we must reconsider the skill set we aim at fostering in our software engineering students, and adapt our intended learning outcomes accordingly. Furthermore, we need to adapt both assessment strategies and the teaching and learning methods we employ, in order to provide our students with a study experience that adheres to the principle of constructive alignment.

**Index Terms**—software engineering education, learning objectives, teaching methods, assessment, large language models

## I. INTRODUCTION

AI-systems that are based on large language models, such as ChatGPT, and large language models trained on code (LLMC) such as Codex [1], have quickly become more powerful over the last year, and are readily available to everybody who can pay for them, including our students. As any new kind of tooling that emerges in a specific domain, ChatGPT and its kin will influence the way that things are done. The multitude of domains that tools such as ChatGPT will influence is highly impressive, if not disconcerting.

For the domain of software engineering (SE), some voices predict that the rise of AI systems will lead to “the end of programming” [2], claiming that future software systems will be trained rather than programmed in a classical way. Even if this is rather speculative, it is undeniable that the availability of ChatGPT offers new possibilities for dealing with typical tasks of the discipline. As a consequence, some traditional skills of the discipline will become less relevant, while at the same time new skill sets will emerge that will be required in future professionals.

Daun and Brings [3] discuss the potential for using generative AIs like ChatGPT in software engineering education. They

argue that the focus needs to shift to assessments that require students to have a more in-depth discussion and propose new results and ideas. Examples are oral exams, group projects and presentations. According to the belief of the authors, the importance of manual coding will decrease and software engineers will need to become more proficient in requirements engineering, design, and testing of generated code. The authors argue that students have to learn to include AI in the software development pipeline. Thus, software engineering education will need to place more emphasis on machine learning, natural language processing and data science [3].

Tu et al. [4] present insights in data science education that are equally relevant to software engineering. The authors argue that LLMs are transforming the landscape from hands-on coding, data wrangling and conducting standard analyses towards assessing and managing analyses performed by LLMs and other generative AIs. Consequently, they place emphasis on cultivating diverse skill sets such as LLM-informed creativity, critical thinking, AI-guided programming, and interdisciplinary knowledge. Furthermore, the authors emphasize the significant role LLMs can play in the classroom, by offering interactive teaching and learning possibilities, i. e. a constructivist learning environment [4].

Lorraine Jacques [5] argues that computer science education can benefit from the lessons learned in math education, which has faced similar challenges. An important point she makes is that the advent of programming assistance tools does not negate the need for programming fluency. Such fluency reduces cognitive load when tackling advanced problems and is crucial in assessing the effectiveness of AI-generated solutions. Jacques proposes a shift in educational focus towards code explanation, experimentation with various approaches, and the utilization of multiple representations, such as UML diagrams, pseudocode, and text. For introductory computer science courses, she suggests keeping the material relevant to students’ interests, while emphasizing the cognitive benefits of learning to code. Additionally, she recommends adapting instructional techniques from math education to the context of computer science courses [5].

In a workshop setting during the SEUH 2023 conference (Software Engineering in Higher Education) held in Paderborn, Germany, in February 2023, co-located with the Software Engineering conference (SE), researchers and profession-

This work was supported by the German Stiftung Innovation in der Hochschullehre (StiL), <https://stiftung-hochschullehre.de/> grant no. FRFMM-455/2022. Thank you for your support.

als in software engineering discussed the impact of ChatGPT on future skills required in software engineers. On this basis, they derived implications on teaching and learning software engineering in the era of ChatGPT and similar systems.

## II. GOALS

If used appropriately, large language models are powerful tools that may be effectively employed in the process of software engineering. In this paper we want to explore the following questions:

- How does the availability of SE tools based on generative AI influence the skill set that professional software engineers will need in the future?
- Which new learning objectives should we address in our teaching to foster these skills in our students?
- In what manner do large language models influence the formal assessment of our students' SE skills?
- What teaching and learning methods are appropriate when teaching SE in a constructively aligned manner in the era of large language models?

## III. LITERATURE REVIEW

A variety of aspects related to SE and SE education in the era of LLMs has been discussed in literature. For the following literature review, we focus on three of these aspects: (A) educational potentials, (B) code generation capabilities, and (C) interaction between LLMs and engineers.

### A. Educational Potentials

In a position paper, Kasneci et al. [6] argue that the potential of large language models (LLMs) like ChatGPT in enhancing learning experiences is substantial. They emphasize the model's excellence in generating summaries and outlines. As a difference from search engines, the provided information is personalized and contingent on the users prompt history.

Consequently, Rasul et al. [7] highlight the potential for an enriched constructivist learning experience. This approach includes personalized tutoring, automated essay grading, and both interactive and adaptive learning modalities. The authors advocate for the integration of LLM APIs into the learning systems of higher education institutions, proposing this as a key component of adaptive learning systems. According to them, educators should be encouraged to consider the benefits of incorporating LLMs into their pedagogical strategies, specifically for creating customized lesson plans. The authors also emphasize the importance of training both students and academics in the effective and ethical use of ChatGPT. This includes guidance on leveraging ChatGPT for innovative assessment strategies, ensuring its responsible application. They underscore the necessity for a paradigm shift in educational priorities, placing a greater emphasis on the responsible and ethical application of generative artificial intelligence. This shift entails the development of innovative assessment strategies that are adapted to the nuances of AI. Critical to this endeavor is addressing inherent biases and the risks of disseminating falsified information through AI systems

[7]. Furthermore, the authors argue for the integration of AI literacy into the skill set required for graduates, recognizing it as essential in an increasingly AI-driven world.

In his comprehensive review, Partha Pratim Ray [8] delves into the various facets of ChatGPT, encompassing its applications, key challenges, ethical considerations, limitations, and future prospects. He identifies five applications within the domain of programming: code generation, code optimization, debugging assistance, code documentation, and code review. He agrees with [7] regarding the role of ChatGPT in personalized learning and tutoring, highlighting its significance in these areas.

### B. Code Generation Capabilities

According to Microsoft's internal research, a significant 55% increase in developer productivity was achieved when using their LLMC, known as Copilot which is based on Codex – a fine-tuned GPT-3.5 [9] model. In their study, 95 professionals participated in a competitive challenge to write an HTTP server in JavaScript. The participants were divided into two groups: one used Copilot, while the other did not. The findings reveal that the group using Copilot not only had a higher success rate – 78% compared to 70% for the non-Copilot group – but also completed the task more quickly. This suggests that Copilot effectively enhances both the efficiency and effectiveness of programming tasks.

Coding test results were published by OpenAI [10] in conjunction with the release of GPT-4.0. According to their report, GPT-3.5 managed to solve 29.3% of easy problems and 0.0% of hard problems on LeetCode, a popular online platform offering programming exercises and coding challenges frequently encountered in technical interviews. In comparison, GPT-4.0 showed an improved performance, solving 75.6% of easy and 6.6% of hard problems from LeetCode.

Furthermore, Bordt and von Luxburg [11] conducted an evaluation of both models on an exam focused on algorithms and data structures. The results indicated a performance level of 51.3% for GPT-3.5 and 60.0% for GPT-4.0, respectively. This comparison highlights the advancements in the capabilities of these language models, particularly in the context of technical problem-solving.

Asare et al. [12] conducted a study exploring the comparative propensity of Copilot and human developers to generate code vulnerabilities. Their findings suggest that Copilot is less prone to introducing the same vulnerabilities as those commonly produced by human programmers. In their experimental trials, Copilot generated only one-third of the vulnerabilities compared to its human counterparts. However, the authors raised a concern regarding the potential for Copilot to mirror the behaviors of large language models (LLMs) in terms of overfitting. This phenomenon, characterized by plain memorization, may lead Copilot to replicate its training data. Such a tendency seems to contradict Microsoft's internal research, which asserts that Copilot reproduces code snippets longer than 150 characters in only about 1% of instances. This discrepancy invites further scrutiny into the mechanisms by

which Copilot processes and generates code, particularly in relation to its training methodology and its ability to innovate beyond mere replication.

Mastropaolo et al. [13] conducted a comprehensive investigation into the sensitivity of Copilot's performance in relation to prompt wording, examining 892 distinct tasks. Their research revealed that variations in the phrasing of prompts describing identical coding tasks lead to differing recommendations, thereby affecting the accuracy of the code generated by Copilot. While Copilot's performance in solving fundamental problems in computer science, like sorting and implementing data structures is impressive [14], it struggles with more complex non-standard problems and with combining multiple methods to generate a solution. Fittingly, the study revealed a relatively low success rate of Copilot, approximately 13%, which can be attributed to the high level of task difficulty, as indicated by an average cyclomatic complexity score of 5.3 (median of 3.0). The study also discovered a significant discrepancy between the types of errors produced: semantic errors were predominant, constituting 73% of the total errors, as opposed to syntax errors, which accounted for a mere 12.5%. Interestingly, the syntax errors were predominantly due to the invocation of non-existent methods, suggesting that while Copilot often generates syntactically correct-looking code, it may lack in semantic accuracy. Moradi [14] et al. concludes that Copilot can become an asset for professionals but also a liability for novice developers who may fail to filter its buggy or non-optimal solutions due to a lack of expertise.

Kürger and Gref [15] studied the performance of different LLMs, including ChatGPT-3.5 and GPT-4.0, within the context of an undergraduate computer science degree program at a university of applied sciences. They found that ChatGPT-3.5 achieved an 79.9% of the maximum possible scores in the ten modules. These modules covered a diverse range of topics including web engineering, distributed systems, data science, operating and real-time systems, software engineering and data network management. GPT-4.0 achieved a slightly better score of 80.2%. The model demonstrated a more consistent overall performance.

Echoing the research focus of [13], [14], Nguyen et al. [16] carried out an empirical study, centering on the understandability of Copilot's outputs across various programming languages. Utilizing 33 questions from LeetCode, they tested Copilot's capabilities in four distinct programming languages. The study found that Copilot successfully generated acceptable solutions for 70% of the questions in at least one of the languages, but with notable disparities: Java (57%), Python (42%), C (39%), and JavaScript (27%). The authors argue that despite these variations, Copilot generally produces code that is understandable, taking into account both cyclomatic and cognitive complexities, across all assessed languages.

In this article, our focus is primarily on literature about Microsoft's Copilot (based on GPT-3.5) though it's important to acknowledge the presence of other sophisticated LLMs such as Duet AI (powered by Google's Gemini [17]) and JetBrains' AI Assistant [18]. One significant challenge in this

field is the limited accessibility of these models, which poses obstacles for researchers conducting studies.

Additionally, it is crucial to highlight the innate capability of these LLMs as few-shot learners [9], which allows for enhanced task-specific performance without the need for additional training. This is achieved by leveraging their ability to integrate external data utilizing their large number of input tokens. A notable example is the utilization of a prompt-based strategy to teach Codex to repair programming errors in multiple languages [19]. This approach capitalizes on few-shot learning to enable Codex to interpret and resolve errors based on messages from compilers, static analyzers, or linters, demonstrating a minimal-effort yet effective application of these models [19].

The results found in existing literature on LLMs and LLMCs vary significantly and are challenging to compare directly. In summary, these models have demonstrated the capability to pass multiple undergraduate-level exams. It is observed that as the complexity of tasks increases, the performance of current LLMs and LLMCs tends to drop significantly. This variation in performance highlights the limitations of these models when dealing with more complex, nuanced tasks. However, further improvements in the LLM- and LLMC-based tools are to be expected.

### *C. Interaction between LLMs and Engineers*

In the context of programming tasks, Kazemitabaar et al. [20] demonstrated that the utilization of Codex significantly enhances code-authoring performance. Importantly, the findings suggest that this enhancement does not come at the expense of skills in manual code modification. Therefore, the authors' research provides evidence that novice learners can effectively employ AI code generators to accomplish tasks without hindering their learning progress. Furthermore, learners with prior experience in using Scratch [21] – a visual programming language developed for children – exhibited notable improvements in their abilities after being introduced to Codex. The study also reveals that individuals with higher pre-existing programming skills derived greater benefits from the use of AI code generators. The study concludes that AI code generators can be a vital tool in reducing frustration among even the most inexperienced users.

To the best of our knowledge, there are only a few empirical studies examining the interaction between programmers and code generation AI, particularly in terms of quantifying the actual performance gains. One small study by [22] compared 24 programmers (with at least two years of experience) in their use of Copilot versus the conventional code completion tool, Intellisense. The study revealed that task completion rates were insignificantly lower with Copilot. Notably, Copilot sometimes led users into time-consuming debugging scenarios due to incorrect code generation. However, tasks completed using Copilot were generally finished faster. The study highlighted potential issues with Copilot, including cognitive overload due to long code generation, an underestimation of bug-fixing efforts, and challenges in using ambiguous comments as spec-

ifications. Participants reported that while Copilot saved time compared to web searches, solutions verified through Stack-Overflow discussions instilled more confidence. The study identified three major hurdles in practical Copilot use: (1) cognitive overload, (2) underestimation of debugging effort, and (3) the unreliability of using comments for specifications.

Barke et al. [23] conducted a similar study investigating the interaction of 20 programmers and Copilot. Their findings mostly agree with [22]. However, their study was not competitive and included tasks that required contributing to an existing codebase. Furthermore, participants had no trouble in understanding generated code, probably because they were more experienced. The main contribution, according to the authors themselves, is the identification of two distinctive modes of interaction, motivated by the thinking fast and slow concept of Kahneman [24]: *acceleration* and *exploration*. The authors argue that in the first mode, programmers already know where they want to go and Copilot helps them to get there quicker. In exploration mode, programmers want to get a starting point and explore their options. Their findings suggest that more experienced programmers spend more time in acceleration mode and have lower expectations of Copilot's ability to solve high-level exploratory tasks.

#### D. Summary

Summing up, our findings from literature suggest that assistant models, like ChatGPT, can enrich a constructivist learning experience enhancing the effectiveness of studying software engineering. They can aid students and educators alike in a wide range of tasks. However, their utilization has to be coupled with the sensitization of ethical consideration, concerns about biases, fairness and transparency, sustainability and the fact that most utilized LLMs are proprietary.

LLMs most likely increase the productivity of professional software engineers. They are especially effective in solving fundamental problems in computer science, but their performance drops with an increase in problem complexity. Even for simple tasks, their results are untrustworthy. However, there is also evidence that they are less prone to introducing the same vulnerabilities as those commonly produced by human programmers.

There is compelling evidence suggesting that programming experience plays a pivotal role not only in effectively leveraging large language models (LLMs) for task-solving, but also in augmenting one's programming capabilities. The concern raised by, e. g., [6] that an overreliance on LLMs might impact the learners' progress was not reproduced by the study carried out by [20]. However, their study is only a first step to understand the effect of LLMs and the outcome significantly depended on the learning environment. Lastly, the abstract differentiation between *acceleration* and *exploration mode* [23] is a useful concept in the context of software engineering education, because each of these modes require a different skill set.

## IV. FUTURE SKILLS IN SOFTWARE ENGINEERS

As AI won't go away in the foreseeable future, SE professionals have to adapt their skill set. Making effective use of the new possibilities of communicating with an untrustworthy but effective artificial partner necessitates, on the one hand, the ability to pose high-quality questions and, on the other hand, the ability to scrutinize the delivered results and critically reflect on their appropriateness and suitability for the task at hand.

Crafting suitable questions or prompts and assessing the coherence of responses demands a robust foundational knowledge in the subject matter; to ask good questions, one has to know what one does not know. To find errors within AI generated code, the importance of syntax becomes less important because studies, such as [14], indicate that LLMs predominantly make semantic errors. Consequently, the muscle memory acquired by programmers over years of practice will become less valuable, since LLMs have an unfair advantage in the mode of *acceleration*.

Consequently, we anticipate a shift in required skills towards effective *exploration*. The ability to efficiently navigate through a myriad of possibilities to find optimal solutions will become paramount. This requires a foundation in abstract, critical and conceptual thinking combined with a broad knowledge enabling engineers to effectively evaluate various starting points. LLMs can then be utilized to refine and target the precise solution. The proficiency to discern and correct erroneous paths taken by LLMs will be crucial. In this emerging landscape, the focus will likely shift from mastering every syntactical detail of programming languages to a broader competence in reading, understanding, and working with large code bases and complex systems across disciplines. Computational and design thinking skills are expected to gain prominence, reflecting the necessity for a deeper understanding of code structure and system behavior. This shift signifies a broader, more holistic approach to software engineering, where the capacity to analyze, conceptualize, and creatively solve problems becomes increasingly vital.

Our literature review suggests that the importance of writing code is likely to diminish and the skill of critically evaluating existing or generated code will become more important – we anticipate another shift from creation to curation. Several SE professionals already utilize LLMs such as Copilot (based on OpenAI's GPT-4 [10]), Duet AI (powered by Google's Gemini [17]), and JetBrains' AI Assistant [18]. These LLMs, seamlessly integrated into integrated development environments (IDEs) like Visual Studio Code and IntelliJ, facilitate various tasks including code generation, summarization, evaluation, refactoring, commenting, management, the generation of commit messages and the detection of vulnerabilities. In addition, LLMs can aid specification generation, just in time developer feedback, testing, and language translation [25].

The integration of multiple, customized LLMs into the development environments of software engineers, particularly those tailored to proprietary codebases of individual compa-

nies, appears to be a possible advancement in the near future. This development will likely necessitate yet another shift in the role of software engineering professionals. They may increasingly participate in activities like labeling and selecting high-quality code during the development process. This active involvement is expected to contribute significantly to the continuous enhancement of the models on which they rely. Such practices would underscore the importance of code curation in software development, as well as in the advancement of specialized LLMs. As a result, engineering professionals would be expected to assume dual responsibilities: firstly, as 'machine-teachers', where they contribute to the training and refinement of LLMs; and secondly, as efficient evaluators of code. In this latter role, they would be responsible for ensuring the quality and relevance of the outputs generated by these models. This dual role represents a significant shift from traditional software engineering tasks, placing a greater emphasis on the interaction with and oversight of AI-driven systems. It suggests a future where software engineers are as much educators and curators of intelligent systems as they are developers of software, highlighting the evolving interplay between human expertise and AI in the field of software development.

However, even if the speed at which the performance of LLMs is getting better, we expect that professionals will not stop at code curation. Professionals will probably continue actively correcting, extending, and writing specific parts of code to combine the parts of a larger system. This code fine-tuning might be supported by LLMs such that professionals steer the writing process in the right direction. LLMs might eventually replace current platforms like StackOverflow [26], which are currently indispensable resources for developers. Nowadays, practitioner software engineers rarely start at clean sheet. This trend will intensify. As highlighted by Daun and Brings [3], requirements engineering and design are set to play a more crucial role in the near future. An indication is the neglect of attention this discipline so far received in the LLM-based software engineering literature [27].

Effectively interacting with fine-tuned large language models necessitates a deep understanding of their specialized capabilities. If AI-generated solutions are flawed or stylistically poor, engineers might inadvertently assimilate suboptimal or incorrect conventions. While this concern is also relevant for web-based examples, personalized solutions gained by communication probably lead novices to perceive the solutions as being more credible. Bender and Koller [28] underscore this hypothesis. They argue that humans tend to presume meaning and communicative intent within a conversation even if there is none. This can amplify the credibility of AI-generated text, and make it more difficult for students to distinguish "between real knowledge and unverified information" [6, p. 9]. Therefore, professionals should be equipped to assess a model's potentials and limitations based on its architecture, training dataset, and size, particularly within the context of their specific discipline. It is important for them to be knowledgeable about various types of models and their

appropriate applications. This requirement is analogous to understanding the diverse ecosystem of operating systems; professionals should similarly comprehend the ecosystem of LLMs. A key aspect of this understanding involves awareness of the varying levels of openness among different models. This encompasses the openness of model weights, training datasets, the architecture and details of the training process, and the implementation code. Such knowledge enables professionals to make informed decisions about which LLMs are best suited to their specific needs and how to effectively utilize them in their work.

Furthermore, future software engineers should be able to foster a sustainable development over the whole lifecycle of products and services. They have to be aware of the tension between AI for sustainability and sustainable AI [29], [30]. This includes the ecological, social and economic aspects of sustainability. It is well-known that the training of LLMs require a lot of resources (energy, hardware, and manual labor) [31]. Moreover, each prediction comes at a cost as well. Engineers have to consider if, when, and where to compute and how to design software in a data driven environment.

Future engineers have to be aware of the model's multiple sources of possible biases, starting from the inductive bias ingrained in the learning algorithm itself. For example, as large language models feed on vast amounts of existing information, they might be used in requirements elicitation as a tool that mirrors historically accumulated user needs and project pitfalls on a general basis.

## V. INTENDED LEARNING OUTCOMES IN THE FACE OF CHATGPT

To prepare students towards the skill set they will need as SE professionals in the era of ChatGPT, we need to adapt the intended learning outcomes of our teaching accordingly. Therefore, software development courses should place more emphasis on code review, analysis and the evaluation of code quality, rather than focusing on the actual coding itself. However, we agree with [5] that programming fluency is still necessary to reduce cognitive overload, but can be achieved in support of LLMs. In software engineering education, the emphasis should shift from coding towards algorithm analysis, software architecture design and requirements engineering in a data driven environment. Students should learn how LLMs are integrated and applied in development environments and reflect the role they fulfill in software engineering.

As well, students require the ability to construct high-quality, effective questions to guide and direct LLMs towards desired outcomes. Learning about certain prompt patterns [32] can be helpful (in the short run), but far more important is the ability to critically evaluate LLM outputs for appropriateness and suitability to specific tasks. Students should also be able to understand and utilize few-shot learning.

Future engineers should cultivate skills for efficient *exploration* through diverse possibilities to identify optimal solutions. Therefore, they need to develop abstract, critical, and

conceptual thinking skills, combined with broad knowledge for effective problem-solving.

Furthermore, a basic level of AI literacy is needed in any software engineer. Large language models need to be demystified. More specifically, students working with large language models should learn the difference between natural language understanding (NLU) and natural language processing (NLP) [28], to be properly aware of the possibilities and limitations of the tools they use. Engineers have to acquire a robust foundational knowledge to, on the one hand, frame relevant questions and prompts for LLMs, but also understand the probabilistic and data-dependent nature of (connectivist) AI. During education, they should gain proficiency in identifying and correcting semantic errors in AI-generated code, with a decreasing focus on syntax.

Students might have to be prepared for dual roles: contributing to the training and refinement of LLMs, and ensuring the quality and relevance of AI-generated outputs. They should learn about the importance and value of high-quality data. However, we are unsure how the second role of engineers will exactly look like, i.e. how much they are involved in machine learning duties. The interplay between code generation, training data curation and model training or fine-tuning will probably be realized by different specialists ranging from software engineers to machine learning experts. We believe that students still have to develop coding skills for ongoing code correction, extension, and writing, complementing the capabilities of LLMs.

Moreover, students should gain comprehensive knowledge about various LLMs, their applications, and how to choose and use them effectively. They must be equipped with skills to incorporate code generation into the software development lifecycle, including the adeptness to deploy, integrate, and refine machine learning models. This is only possible if they understand the levels of openness in different models, including model weights, training datasets, architecture, and implementation code.

In addition, as educators we have to support students to foster sustainable development practices, considering ecological, social, and economic aspects. Future engineers have to be aware of the resource requirements of LLMs and their environmental impact. Furthermore, students should be sensible towards potential biases in LLMs and their implications in software development.

To gain a clearer notion of what competences we aim to foster in our students through our teaching efforts, we specify intended learning outcomes on different levels of expertise, in accordance to Bloom's revised taxonomy [33] as visualized in Fig. 1.

Table I specifies a first selection of intended learning outcomes that we need to address in our teaching, in order to help prepare our students for using generative AI within the software engineering process in a purposeful way.

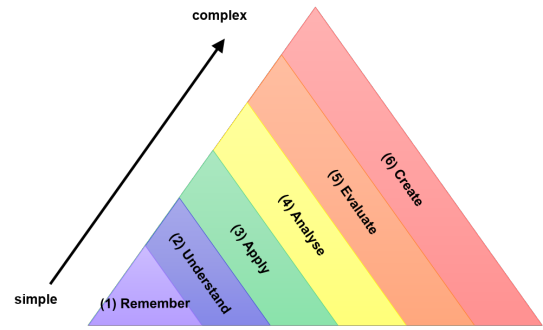


Fig. 1. Competence levels according to Bloom's revised taxonomy [33].

## VI. ASSESSING SE SKILLS IN THE ERA OF LLMs

When shifting our focus towards assessing those skills that are newly required in the era of LLMs, educators should clarify and clearly communicate to students which practices constitute cheating or plagiarism – and which kinds of usage of LLMs are in fact desirable as being part of emerging professional practices. In parallel, we need to devise new task types or even assessment formats to properly address the skill sets we previously defined as intended learning outcomes.

LLMs are especially proficient in solving training exercises for programming novices [34], [35]. Educators can only mitigate this problem by artificially detouring from the LLM training dataset using customized code. One solution, pointed out by Daun and Brings [3], is to shift to oral exams, group projects and presentations to foster in-depth discussion which are needed for performance evaluation.

The gaining importance of code review has to be reflected in the assessment of software engineering skills. Students should curate, explain and evaluate a body of code (both AI-generated and human-written) for quality and relevance. Since the output of LLMs is contingent, educators should curate (realistic) code themselves beforehand. Provided with the code and additional explanations, students have to rank and evaluate the code in an offline setting, e.g. oral exams or group discussions. As suggested by [5], assignments can be designed to include various tasks that involve explaining and manipulating code, translating it into different representations, testing the generated code with various inputs, and evaluating its efficiency.

Assessing *exploration* and computational thinking skills could be achieved by open-ended projects that require students to explore multiple solutions using LLMs and select the best one based on conceptual reasoning. The students' exploration can be documented and presented via project-based assignments. Within this context, there should be a strong emphasis on the explanation of sub-solutions. Another option is to encourage experimentation with various starting points in a limited timeframe, some of which may lead to deadlocks. This approach differs significantly from traditional methods, as it focuses on the ability to navigate through potential impasses and explore diverse solutions. The challenge lies in effectively

TABLE I  
INTENDED LEARNING OUTCOMES FOR USING GENERATIVE AI IN SOFTWARE ENGINEERING

| L              | Students ...                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (1) Remember   | <p>correctly name ...</p> <p>... the fundamental principles of code review, analysis, and quality evaluation.</p> <p>... key prompt patterns for effective LLM interaction.</p> <p>... techniques for <i>exploration</i> and problem-solving in software engineering.</p> <p>... the differences between NLU and NLP.</p> <p>... existing tools based on generative AI that can be employed in the context of software engineering.</p> <p>... the environmental impact and resource requirements of LLMs.</p>                                                                                                                                                                                                                                                                                                                                       |
| (2) Understand | <p>explain in their own words ...</p> <p>... the role and importance of programming fluency in supporting LLMs.</p> <p>... the process of formulating questions for LLMs.</p> <p>... the importance of critical and conceptual thinking in engineering.</p> <p>... the probabilistic and data-dependent nature of machine learning approaches.</p> <p>... the levels of openness in different LLM models.</p> <p>... the potential biases in LLMs and their implications.</p> <p>... how tools based on generative AI can facilitate different stages of the SE process.</p> <p>... possible limitations of tools based on generative AI with respect to a possible usage within the SE process.</p> <p>... for which parts of a problem-solving process, or a solution that is to be developed, using a generative AI will be helpful, and why.</p> |
| (3) Apply      | <p>... schematically employ generative AI to generate code snippets for standard cases, such as data access.</p> <p>... use LLMs to assist in software engineering tasks like algorithm analysis and software architecture design, following a well-defined action pattern.</p> <p>... systematically employ <i>exploration</i> skills to identify optimal solutions in engineering tasks, following a well-defined procedure.</p> <p>... frame questions and prompts for LLMs according to well-defined patterns.</p> <p>... schematically integrate LLMs into the software development lifecycle</p> <p>... implement sustainable development practices in LLM-related projects, following well-defined patterns and procedures.</p>                                                                                                               |
| (4) Analyze    | <p>... examine diverse problem-solving approaches and their effectiveness.</p> <p>... detect semantic errors in AI-generated code.</p> <p>... investigate the effectiveness of different LLMs in specific scenarios.</p> <p>... reflect on the social, ecological, and economic aspects of LLM usage.</p> <p>... identify from a given design document which parts of the system can be developed by using a generative AI.</p> <p>... determine what a generated artefact, such as a model or a documentation, actually expresses.</p> <p>... determine what a generated source code actually does in terms of functionality and created effect.</p>                                                                                                                                                                                                |
| (5) Evaluate   | <p>... differentiate between high and low-quality of both human and AI-generated code.</p> <p>... identify strengths and weaknesses of a generated artefact.</p> <p>... judge LLM outputs for their appropriateness and suitability.</p> <p>... assess the effectiveness of LLMs in various software development environments.</p> <p>... judge the effectiveness of few-shot learning in different contexts.</p> <p>... compare the outcomes of different <i>exploration</i> strategies.</p> <p>... judge the limitations and possibilities of LLM tools.</p> <p>... validate the effectiveness of code corrections and extensions in LLMs.</p> <p>... take the ethical aspects in software development involving LLMs into account.</p> <p>... critically assess generated artefacts with respect to established quality criteria.</p>             |
| (6) Create     | <p>... construct high-quality, effective questions for LLMs in practical scenarios.</p> <p>... design projects that incorporate LLMs into software development processes.</p> <p>... develop new strategies for interacting with LLMs through innovative questioning techniques.</p> <p>... formulate new methods for problem-solving that combine critical thinking and <i>exploration</i>.</p> <p>... innovate new approaches to integrate AI literacy into software engineering practices.</p> <p>... develop new models for integrating coding skills and LLM capabilities.</p> <p>... design new systems for effectively choosing and using LLMs in software development.</p> <p>... develop new strategies for fostering ethical and sustainable use of LLMs in software engineering.</p>                                                      |



simulating and evaluating these scenarios to accurately gauge the students' exploration skills.

Assessing the ability to ask good questions and think critically could be achieved by using case studies where students interact with LLMs to solve specific problems. In this field, computer science educators might need support from other disciplines, e. g. linguist and communication scientists. Basic writing tests could evaluate the students' ability in deductive and inductive reasoning. The same holds true for AI literacy and the LLM ecosystem. However, interactive tests can and should be employed. Specifically, we need formats that interactively test the students ability to probe a machine learning model – its capabilities, limitations and biases.

For example, students could be task to trick an AI into errors or unwanted behaviors. One example would be the challenge to trigger hallucination via a prompt, or to fool an image classifier based on the provided knowledge about its training data. In any case, both the assessment and the teaching should be practical, which can be achieved by employing interactive machine learning – a paradigm engaging people in a tight interaction loop with a learning algorithm [36].

The problem of assessing sustainability competencies is broadly discussed in literature, and is out of the scope of this article. We refer to [37] for further reading. That said, the assessment should include topics most relevant for software engineers, such as sustainable computing and sustainable AI.

Changing perspective, we educators could employ ChatGPT to generate ideas for exercises or exam tasks. As well, some colleagues already discuss using ChatGPT for correcting assignments and grading student work. However, in this context a host of issues of correctness, responsibility as well as legal aspects still needs to be resolved beforehand.

## VII. TEACHING AND LEARNING METHODS

In traditional teaching of software development, teaching and learning processes followed the notion that competences on the different skill levels develop according to the levels' dependencies, as depicted in Figure 2.

Skills on levels (1) Remember and (3) Apply focus on fact knowledge (1) and actions that are executed according to well proven step-by-step patterns (3). Both can be acquired by diligence. In contrast to this, developing the skill level stack on the right side of Figure 2 requires a variety of complex cognitive competences, such as abstract thinking, critical thinking or the ability to take decisions. Usually, skills on levels 2 and 3 emerge hand in hand, in the sense that actively applying and executing a concept or procedure increases the understanding of the issues at hand [38].

In professional practice, every system we develop is somewhat unique. Therefore, we usually face development challenges that cannot be solved schematically, thus requiring skills on level (6) Create. So obviously, this is the level that we aim at when fostering our students' skills.

According to the notion of DePhillips et al. [39, p. 69], learning paths pass through a sequence of different stages, as

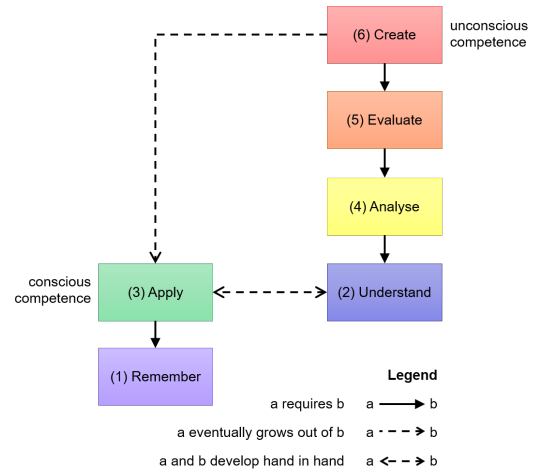


Fig. 2. Dependencies of competence levels within the learning process [38].

depicted in Figure 3. The rightmost stage, unconscious competence, corresponds to level (6) Create of Bloom's revised taxonomy.

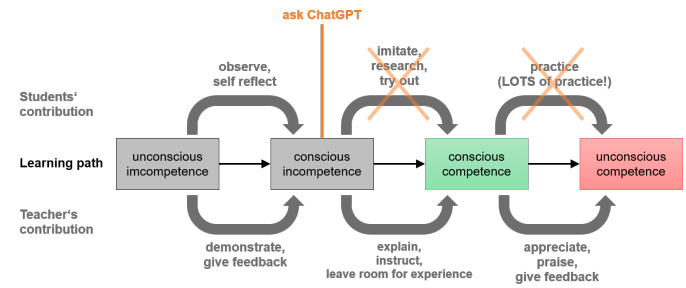


Fig. 3. Different stages of competence along the learning process, and impact of relying excessively on ChatGPT.

Having reached the stage of conscious incompetence, moving on along the learning path tends to require a reasonable amount of effort on the student's side. We observe that many students shy of investing this effort, especially if things get challenging. As a consequence, especially in the face of approaching deadlines, we observe in our students a tendency to resort to "ask ChatGPT" when things get rough.

As students in fact *will* need well founded expertise in interacting with LLM based tools, asking ChatGPT is not a bad thing in principle. However, we need to ensure that students will develop as much base expertise e. g. in the craft of coding, as they will need later on to critically assess AI-generated artifacts. Therefore, the teaching and learning methods we employ have to ensure that students acquire a reasonable selection of basic skills as well.

As mentioned above, the skill of writing code will be less important in future SE professionals, whereas code review skills will become highly significant. Programming is an intertwined process of reading and writing code. Without understanding syntax and semantics one will be unable to work



with generated code. However, assuming the code is correct, findings of a small study [40] suggest that understanding the code's intention from the text surface is a major challenge in reading. Since LLMs can offer explanations along side their generated code and the intention is given by the engineer, code reading might actually drastically accelerate the learning progress towards the ability to review and manipulate given but also explained code. Further research is required to answer the question if, under these circumstances, active coding starting from a clean sheet is still necessary. In general, we have to answer the urgent question if it possible to learn how to review and manipulate code without being engaged in writing code from more or less of a clean sheet.

We believe that computational thinking [41], [42] should be the focus of programming courses, while keeping programming fluency alive utilizing LLMs. Educators should focus on language independent concepts and foster multiple representations including physical ones. For instance, numerous sorting algorithms can be practically demonstrated and conceptually understood by manually performing them with a physical deck of cards.

Game-based learning, implemented through a custom software library, can be an effective method to maintain engagement with fundamental algorithms. For instance, in our Computational Thinking course, we use roboworld [43], a Python package inspired by Kara [44]. In this setup, students are tasked with maneuvering a robot in a two-dimensional world. The robot is designed with limited functionality, allowing students to focus on solving fundamental problems. Additionally, this package provides instant visual feedback, enhancing the learning experience. An example of this can be seen in Figure 4. This hands-on approach not only makes learning more interactive but also helps in solidifying the understanding of core computational concepts.

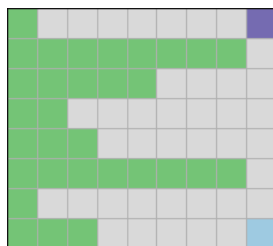


Fig. 4. Example of a roboworld sorting exercise. The goal is to sort the green rows, representing a number, in an ascending order according to their length.

With the growing amount of excellent online sources and the support of LLMs, learning in a flipped classroom environment becomes increasingly effective. Self-directed learning and flexibility can be effectively supported by providing high-quality online resources for students to learn software engineering concepts on their own pace, reserving classroom time for discussions, practical exercises, and collaborative work. Furthermore, we expect that learning methods such as project-based, inquiry-based, reflective and collaborative

learning, where the lecturer takes the role of a facilitator or mentor, can be more effectively employed using LLMs.

The focus on *exploration* could be realized by small research projects where students present and discuss their result with their peers and the instructors. Students should explain both their own code and code from others to their peers and the instructors.

Discussions and debates focusing on key topics such as AI ethics, the future of software engineering with LLMs, the sustainability of AI, and its impact on society should be organized. In addition, it is important to 'demystify' the workings of LLMs. As suggested, teaching can include interactive machine learning elements, as defined by [36]. This approach would allow students to actively engage with the AI, provoking and identifying instances where it might produce hallucinations or be tricked into making false predictions. Such practical experiences not only enhance understanding, but also provide insights into the limitations and capabilities of AI systems.

The potential of personalized constructivist learning experiences can be greatly enhanced by utilizing LLMs. They have the capability to generate personalized exercises quickly, making it increasingly important to monitor each student's individual learning progress, which allows educators to provide the most effective exercises or challenges tailored to each student's needs.

In addition to leveraging LLMs for personalized and active learning, educators should also focus on teaching students how to learn effectively in such an environment, leading to continuous self-assessment. However, this process is not unidirectional; it requires a collaborative effort. Educators must be open to learning from their students, especially in these early stages of the LLM era. Therefore, while this article primarily addresses the skills required for future software engineers, it is equally important for educators to evolve their skill sets to adapt to these new teaching and learning methodologies.

## VIII. CONCLUSIONS

LLM-based tools such as ChatGPT will significantly change how IT solutions are created, and thus how SE professionals work. We need to adapt our teaching to accomodate these changes in an adequate way, and to explicitly foster those skills that our students will need in their professional future.

All in all, we expect a shift of required skill levels towards the higher levels according to Bloom's revised taxonomy. Specifically, students will require profound skills on levels (4) Analyze and (5) Evaluate, in combination with the ability to efficiently process large amounts of (generated) artifacts, in order to be able to process and properly assess the suitability of the AI generated results for the context at hand.

To what extent "traditional" SE and programming expertise will still be necessary as a basis for these evaluation skills needs further clarification. Furthermore, we need to develop teaching and learning settings that foster this necessary basis in our students themselves – in spite of the availability of AI-systems that would offer short cuts.

## REFERENCES

- [1] M. Chen et al., “Evaluating large language models trained on code,” 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [2] M. Welsh, “The end of programming,” *Commun. ACM*, vol. 66, no. 1, p. 34–35, dec 2022.
- [3] M. Daun and J. Brings, “How ChatGPT will change software engineering education,” in *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*, ser. ITiCSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 110–116.
- [4] X. Tu, J. Zou, W. J. Su, and L. Zhang, “What should data science education do with large language models?” 2023. [Online]. Available: <https://arxiv.org/abs/2307.02792>
- [5] L. Jacques, “Teaching CS-101 at the dawn of ChatGPT,” *ACM Inroads*, vol. 14, no. 2, pp. 40–46, may 2023.
- [6] E. Kasneci et al., “ChatGPT for good? On opportunities and challenges of large language models for education,” *Learning and Individual Differences*, 2023.
- [7] T. Rasul, S. Nair, D. Kalendra, M. Robin, F. de Oliveira Santini, W. J. Ladeira, M. Sun, I. Day, R. A. Rather, and L. Heathcote, “The role of ChatGPT in higher education: Benefits, challenges, and future research directions,” *Journal of Applied Learning & Teaching*, vol. 6, no. 1, pp. 41–56, 2023.
- [8] P. P. Ray, “ChatGPT: A comprehensive review on background, applications, key challenges, bias, ethics, limitations and future scope,” *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 121–154, 2023.
- [9] Tom B. Brown et al., “Language models are few-shot learners,” 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [10] OpenAI, “GPT-4 technical report,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [11] S. Bordt and U. von Luxburg, “ChatGPT participates in a computer science exam,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.09461>
- [12] O. Asare, M. Nagappan, and N. Asokan, “Is GitHub’s Copilot as bad as humans at introducing vulnerabilities in code?” 2023. [Online]. Available: <https://arxiv.org/abs/2204.04741>
- [13] A. Mastropaolo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, and G. Bavota, “On the robustness of code generation techniques: An empirical study on GitHub Copilot,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 2149–2160.
- [14] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. J. Jiang, “GitHub Copilot AI pair programmer: Asset or liability?” *Journal of Systems and Software*, vol. 203, p. 111734, 2023.
- [15] T. Krüger and M. Gref, “Performance of large language models in a computer science degree program,” 2023. [Online]. Available: <https://arxiv.org/abs/2308.02432>
- [16] N. Nguyen and S. Nadi, “An empirical evaluation of GitHub Copilot’s code suggestions,” in *2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*, 2022, pp. 1–5.
- [17] Google DeepMind, “Gemini: A family of highly capable multimodal models,” 2023. [Online]. Available: <https://deepmind.google/technologies/gemini/>
- [18] JetBrains, “Ai assistant,” 2023. [Online]. Available: <https://plugins.jetbrains.com/plugin/22282-ai-assistant>
- [19] H. Joshi, J. Cambronero Sanchez, S. Gulwani, V. Le, G. Verbruggen, and I. Radiček, “Repair is nearly generation: Multilingual program repair with LLMs,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, pp. 5131–5140, Jun. 2023.
- [20] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman, “Studying the effect of AI code generators on supporting novice learners in introductory programming,” in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’23. New York, NY, USA: Association for Computing Machinery, 2023.
- [21] M. Resnick, J. Maloney, A. Monroy-Hernández, N. Rusk, E. Eastmond, K. Brennan, A. Millner, E. Rosenbaum, J. Silver, B. Silverman, and Y. Kafai, “Scratch: Programming for all,” *Commun. ACM*, vol. 52, no. 11, p. 60–67, nov 2009.
- [22] P. Vaithilingam, T. Zhang, and E. L. Glassman, “Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models,” in *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*, ser. CHI EA ’22. New York, NY, USA: Association for Computing Machinery, 2022.
- [23] S. Barke, M. B. James, and N. Polikarpova, “Grounded Copilot: How programmers interact with code-generating models,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, apr 2023.
- [24] D. Kahneman, *Thinking, Fast and Slow*. London: Penguin, 2011.
- [25] I. Ozkaya, “Application of large language models to software engineering tasks: Opportunities, risks, and implications,” *IEEE Software*, vol. 40, no. 3, pp. 4–8, 2023.
- [26] “Stack overflow,” 2008 Accessed: 2024-01-04. [Online]. Available: <https://stackoverflow.com/>
- [27] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, “Large language models for software engineering: Survey and open problems,” 2023.
- [28] E. M. Bender and A. Koller, “Climbing towards NLU: On meaning, form, and understanding in the age of data,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, Jul. 2020, pp. 5185–5198.
- [29] A. van Wynsberghe, “Sustainable AI: AI for sustainability and the sustainability of AI,” *AI and Ethics*, vol. 1, no. 3, pp. 213–218, feb 2021.
- [30] R. Vinuesa, H. Azizpour, I. Leite, M. Balaam, V. Dignum, S. Domisch, A. Felländer, S. D. Langhans, M. Tegmark, and F. F. Nerini, “The role of artificial intelligence in achieving the sustainable development goals,” *Nature Communications*, vol. 11, no. 1, Jan. 2020.
- [31] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” 2019. [Online]. Available: <https://arxiv.org/abs/1906.02243>
- [32] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, “A prompt pattern catalog to enhance prompt engineering with ChatGPT,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.11382>
- [33] L. W. Anderson, D. R. Krathwohl, P. W. Airasian, K. A. Cruikshank, R. E. Mayer, P. R. Pintrich, J. Raths, and M. C. Wittrock, *A Taxonomy for Learning, Teaching, and Assessing. A Revision of Bloom’s Taxonomy of Educational Objectives*, 1st ed. New York: Longman, 2001.
- [34] J. Finnie-Ansley, P. Denny, B. A. Becker, A. Luxton-Reilly, and J. Prather, “The robots are coming: Exploring the implications of OpenAI codex on introductory programming,” in *Australasian Computing Education Conference*. ACM, feb 2022.
- [35] J. Berrezueta-Guzman and S. Krusche, “Recommendations to create programming exercises to overcome ChatGPT,” in *2023 IEEE 35th International Conference on Software Engineering Education and Training (CSEE&T)*, 2023, pp. 147–151.
- [36] S. Amershi, M. Cakmak, W. B. Knox, and T. Kulesza, “Power to the people: The role of humans in interactive machine learning,” *AI Magazine*, vol. 35, no. 4, pp. 105–120, 2014.
- [37] A. Annelin and G.-O. Boström, “An assessment of key sustainability competencies: a review of scales and propositions for validation,” *Int. Journal of Sustainability in Higher Education*, vol. 24, 2022.
- [38] V. Thurner, P. Chavarroche, A. Böttcher, and D. Zehetmeier, “A concept for an intelligent tutoring system to support individual learning paths in software development courses,” in *Teaching and Learning in a Digital World*, M. E. Auer, D. Guralnick, and I. Simonics, Eds. Cham: Springer International Publishing, 2018, pp. 774–783.
- [39] F. A. DePhillips, W. M. Berliner, and J. J. Cribbin, *Management of Training Programs – Unit 2: Principles of Learning and Training*. Homewood, Illinois: Richard D. Irwin, Inc., 1960.
- [40] T. Busjahn and C. Schulte, “The use of code reading in teaching programming,” in *Proceedings of the 13th Koli Calling International Conference on Computing Education Research*. New York, NY, USA: Association for Computing Machinery, 2013, p. 3–11.
- [41] J. M. Wing, “Computational thinking,” *Commun. ACM*, vol. 49, no. 3, p. 33–35, Mar. 2006.
- [42] V. J. Shute, C. Sun, and J. Asbell-Clarke, “Demystifying computational thinking,” *Educational Research Review*, vol. 22, pp. 142–158, 2017.
- [43] B. Zönnchen, “roboworld,” 2022 Accessed: 2024-01-04. [Online]. Available: <https://robo-world-doc.readthedocs.io/en/latest/index.html>
- [44] R. Reichert, J. Nievergelt, and W. Hartmann, *Programmieren mit Kara*. Springer-Verlag, 2005.